INNOPOLIS BIO INNOVATIONS · PLM PLATFORM

Goods Receipt — Frontend Change Guide

Frontend team · Backend v1.0.2 · Partial goods receipt: send the per-delivery quantity

Audience: Frontend team · **Backend version:** 1.0.2 · **Date:** 2026-07-13

This is what you need to change so partial goods receipt works correctly. It directly addresses the "40 then 20 showed 80 instead of 40" issue.

TL;DR — the one rule

> **On each receipt, send the quantity that arrived in THIS delivery (the delta) — NOT the running total.** The backend now accumulates it for you.

That single change fixes the balance bug. Everything else below is detail.

What changed on the backend

Previously the receive call **overwrote** the line's received quantity, so a second delivery replaced the first instead of adding to it — that's why 40 then 20 showed a remaining of 80.

Now `POST /api/purchase-orders/:id/receive` **accumulates**: the accepted quantity of each delivery is added onto the line, and each delivery is stored as a **Goods Receipt Note (GRN)**. Requires the **v1.0.2** backend to be deployed.

Your scenario, fixed (order = 100)

Step	You send `received_qty`	Line `received_qty` (cumulative)	Remaining (`100 − received`)	On-hand stock
Receipt 1	**40**	40	**60**	40
Receipt 2	**20**	**60**	**40** ✅	60

After 40 then 20 → **received 60, remaining 40** (not 80). ✅

⚠️ If the UI sends the cumulative value on receipt 2 (i.e. `60` instead of `20`), the backend adds it again → wrong. **Always send just the newly-arrived quantity.**

The request

`POST /api/purchase-orders/:id/receive` — roles: Purchase, Stores

```
{
  "warehouse_id": "<uuid>",          // required — where accepted stock is booked
  "note": "Courier LR-88213",         // optional (per delivery)
  "lines": [
    {
      "po_line_id": "<uuid>",
      "received_qty": 20,             // arrived in THIS delivery (delta, not total)
      "rejected_qty": 0,              // optional, default 0
      "batch": "LOT-2026-07"          // optional
    }
  ]
}
```

Rules (surface these in the UI):

- The PO must be `Open` or `Partially Received` to receive.
- A `po_line_id` may appear **only once** per request.
- Accepted qty (`received_qty − rejected_qty`) is what accumulates and enters stock. **Cumulative accepted may not exceed the ordered qty** → `400` .
- At least one line must have `received_qty > 0` .

The response

`data` is the updated PO with its lines. A line's **`received_qty` is the cumulative accepted quantity** and drives `received_pct` / `status`:

```
{
  "success": true,
  "message": "Goods receipt recorded",
  "data": {
    "id": "<uuid>", "number": "PO-0007",
    "status": "Partially Received",   // → "Received" when every line is fully received
    "received_pct": "60.00",
    "lines": [
      { "id": "<uuid>", "part_number": "MB-VA-15-3040",
        "quantity": "100.000",        // ordered
        "received_qty": "60.000" }     // cumulative accepted
    ]
  }
}
```

Error envelope: `{ "success": false, "error": "..." }` with HTTP `400`. Common ones: over-receipt (`accepted quantity … exceeds the remaining quantity (40)`), duplicate line, `warehouse_id does not exist`, wrong PO state.

What you have to do (checklist)

- [ ] **Send the per-delivery quantity**, not the cumulative total. Label the input "Received now" / "Receiving in this delivery".
- [ ] **Compute remaining = `quantity − received_qty`** from the response and show it; pre-fill "Received now" with the remaining and cap input at it.
- [ ] Keep the **Receive** action enabled while `status` is `Open` or `Partially Received`; disable it once `Received`.
- [ ] After each receipt, **re-read the PO** (the response already returns the updated PO+lines) and refresh remaining / status.
- [ ] Handle the `400` errors inline (over-receipt, duplicate line, bad warehouse) using the `error` message.
- [ ] Parse numeric fields — they come back as **strings** (e.g. `"60.000"`).
- [ ] (Nice to have) Show the **delivery history** — see below.

Delivery history (optional but recommended)

`GET /api/purchase-orders/:id/receipts` → one record per delivery, newest first:

```
[
  { "grn_number": "GRN-0002", "received_at": "2026-08-01T09:12:00Z",
    "received_by_name": "Raj Patel", "note": "Balance lot",
    "lines": [ { "part_number": "MB-VA-15-3040",
                 "received_qty": "20.000", "rejected_qty": "0.000",
                 "accepted_qty": "20.000", "batch": "" } ] }
]
```

Render it as a "Receipts / Deliveries" panel on the PO page so users can see each partial delivery (who, when, how much, which lot).

Rejections (edge case)

If a delivery has rejected units, only the **accepted** part (`received_qty − rejected_qty`) counts toward the order and enters stock. Rejected units **do not** close the line — you can receive a replacement later. So `remaining = ordered − cumulative accepted`.

How to confirm it's working

Create a PO for 100, then:

1. Receive `40` → response shows line `received_qty: "40.000"`, `status: "Partially Received"`; remaining you compute = 60.
2. Receive `20` → line `received_qty: "60.000"`; remaining = **40**. ✅
3. Try to receive `50` → `400` (exceeds remaining 40).
4. Receive `40` → line `received_qty: "100.000"`, `status: "Received"`.

If step 2 shows 60 received / 40 remaining, you're wired correctly. If it shows 20 received / 80 remaining, the UI is still sending the cumulative value — send the delta instead.

Full contract: `FRONTEND_API_GUIDE.md` *§7. Full release detail:* `RELEASE_NOTES_1.0.2`.